Preprint · October 2026 · https://verdikt-ai.github.io/verdikt/pco · CC BY 4.0

# Proof-Carrying Operation: Device Agents Whose Completion Is Checked, Not Declared

Hussein Abdinoor Mohamed and Kedar R. Haldankar · Verdikt


## Abstract

Agents that operate phones decide for themselves when a task is done, and most of their failures happen at that decision: they stop before every part of a multi-step goal is finished, give up when evidence is hard to find, or report facts they never checked. We move the decision from the model to a checker. In *proof-carrying operation* (PCO) the agent compiles the goal into typed obligations (state changes to make, facts to report, results to derive, conditions to resolve) and closes each with the value it claims and the recorded screen that shows it; a deterministic checker accepts a claim only if that screen's UI tree contains it, challenges claims of absence, and refuses to end the episode while obligations are open, resuming the same session with the same action budget. The final answer is completed from proven facts, and every run leaves a certificate. On iOSWorld, PCO raises performance on all 133 tasks from 77 to 90 fully passed (57.9% to 67.7%) and the mean rubric score from 0.857 to 0.885 (exact McNemar $p = 0.007$); an independent repeat reproduces it (91/133, 0.890), without app-specific knowledge or tuning on the test tasks. An ablation attributes the gain to the complete, evidence-bound obligation ledger rather than to sending the agent back to work. On AndroidWorld, which is near its ceiling, the same operator averages 95.1% pass@1 over three suite seeds with no app knowledge, and executable goal contracts do not change that (95.4%).


## 1 Introduction

Mobile-agent benchmarks now report results above 95% on AndroidWorld [1], and several systems claim 100%. The published 100% results are not single-run pass@1 under the official step limits: one system's 116 published passing trajectories span 25 dates over six months [4], and another refines its app notes and memory between evaluation rounds on the benchmark itself [5]. On iOSWorld [2], whose multi-app and personal-memory tasks leave room, our agent's failures were rarely about capability. Of its first 40 imperfect tasks only 6 ran out of actions; the rest stopped early, skipped a sub-goal, decided a fact did not exist (in one case because it never opened the reply threads where the fact was), or left a requested fact out of the answer.

The common weakness is that the agent is the judge of its own completion. In earlier work on coordinating episodic agents we argued for the invariant that *an agent never certifies its own work*, enforced in the Orcha system by a human-only verification gate [13]. Here we carry that invariant into device operation, where much of the outcome is visible on screen and can be checked mechanically. Our contributions:

1. **Proof-carrying operation**: typed obligations, claims checked against recorded UI evidence, absence challenges, a completion gate that resumes the same session without resetting its budget, answers assembled from proven facts, and a per-run certificate.
2. Evidence about what drives the effect: a first version (v1) had no measurable effect; two changes to how the goal is compiled (obligations for derived results, clause-by-clause compilation, cost-aware proving) produced a significant, reproducible gain (v2); and an ablation shows that the completion gate's resumption rounds add little on top of the ledger at a 50-action budget.
3. An operator design (act-then-observe, identity-pinned targeting, focus-verified typing) that averages 95.1% pass@1 on AndroidWorld over three suite seeds with no app knowledge.
4. Paired statistics, every trajectory published as a replay, and an open-source release (Verdikt Core, Apache-2.0).

## 2 Related work

**Benchmarks.** AndroidWorld [1] has 116 parameterised tasks with programmatic success checks; iOSWorld [2] has 133 tasks over 26 apps graded by a rubric judge.

**Verification in agents.** AdaPlanner [6] has an LLM write assertions in a code plan for text environments. WebTestPilot [7] has an LLM write pre- and postconditions in a DSL for web testing from a given specification. VeriSafe Agent [8] turns an instruction into a logical specification over developer-written predicates and gates critical actions. Surfer 2 [9] and AdecPilot [10] gate the end of an episode on a model's judgment, and Mobile-Agent-E [11] reflects on screen changes with a model. Minitap [4] decomposes goals into subgoals tracked by an orchestrator and verifies text input. In each, either the checks come from someone other than the operating agent or completion is judged by a model. PCO combines agent-authored claims checked deterministically against the live accessibility tree, with no app changes, an absence challenge, and answers that may only add proven facts.

**Coordination of agent work.** Orcha [13] coordinates episodic agents through durable, audited state: typed request and task state machines, agents restored from self-written memory digests, a human-only verification gate and authority keyed to reversibility. PCO adopts its non-self-certification invariant and typed, auditable state, and replaces the human verifier with a mechanical one for outcomes visible in the UI.

## 3 The operator

One general model (Claude Opus) operates the device through Verdikt's tools. **Act-then-observe:** every action returns the screen it produced. **Identity-pinned targeting:** an element index refers to the screen the agent saw, and the element is re-found on the live screen by type, identifier and label before acting, so a list that loads or a keyboard that opens cannot redirect a tap; in the shipped product this fixed a reproducible mis-tap. **Focus-verified typing:** a field is tapped until it reports focus, and typed text is read back.

## 4 Proof-carrying operation

The name borrows from proof-carrying code [12]: the run ships with a certificate that a simple checker verifies.

**Algorithm 1.** Proof-carrying operation (v2)

1. **Compile.** Before acting, go through the goal clause by clause and record every obligation: `DO` (a state change, one per recipient or item; "confirm X" is a DO proven by the screen showing X), `KNOW` (a fact to report or use), `SAY` (a result to derive: a total, comparison, pattern, recommendation) and `IF` (a condition whose answer is itself a fact).
2. **Budget.** Allocate the action budget across obligations, costliest first; with few actions left, finish open DOs before anything else.
3. **Discharge.** Close an obligation while its proof is on screen (closing costs no action), with the claimed value copied from the screen. Never spend actions returning to a screen only to prove something. A SAY is closed with the derived result.
4. **Check.** A DO or KNOW claim is accepted only if the cited screen's UI tree contains it (normalised match; long text by 80% token coverage). A SAY must be stated and goes into the answer. A claim of absence is challenged once.
5. **Gate.** While obligations are open, rejected or unchallenged-absent, resume the same agent session on the same device session with the same action budget (at most two rounds).
6. **Answer.** Complete the final answer with every proven KNOW and SAY it left out; never add an unproven one.
7. **Certify.** Emit the ledger: each obligation's type, verdict, claim and evidence step.

The ledger is a typed state machine in the sense of [13]: each obligation moves from open to proven, unproven or absent, and only the checker moves it to proven; the agent's claim of completion is a request for verification, not the verdict. The checker contains no app-specific knowledge.

**From v1 to v2.** Version 1 had DO, KNOW and IF only. Its ledgers were well kept (most obligations proven) but incomplete: open-ended requests such as "correlate", "recommend" or "summarise" were never recorded, so the checker could not ask for them, and the agent spent actions returning to screens to prove claims, running out of budget on long tasks. Version 2 adds SAY obligations, clause-by-clause compilation and cost-aware proving.

## 5 Experimental setup

**iOSWorld:** 133 tasks on iPhone 17 Pro simulators (iOS 26.5), five in parallel, driven by the Verdikt iOS session; iOSWorld's own data reset and reseed before each task; a 50-action budget and a 30-minute limit; graded by iOSWorld's unchanged rubric judge run with Claude Sonnet 4.6. A task is fully passed when every rubric criterion is satisfied. **AndroidWorld:** the official 116-task suite, official step budgets (10 × complexity) and grader, one attempt per task, Pixel 6 profile, arm64 Android 13 emulator. **Statistics:** Wilson intervals for pass rates, bootstrap intervals for mean rubric scores, exact McNemar tests on paired pass/fail outcomes and sign tests on paired rubric scores, all on the same tasks.

## 6 Results

### 6.1 iOSWorld

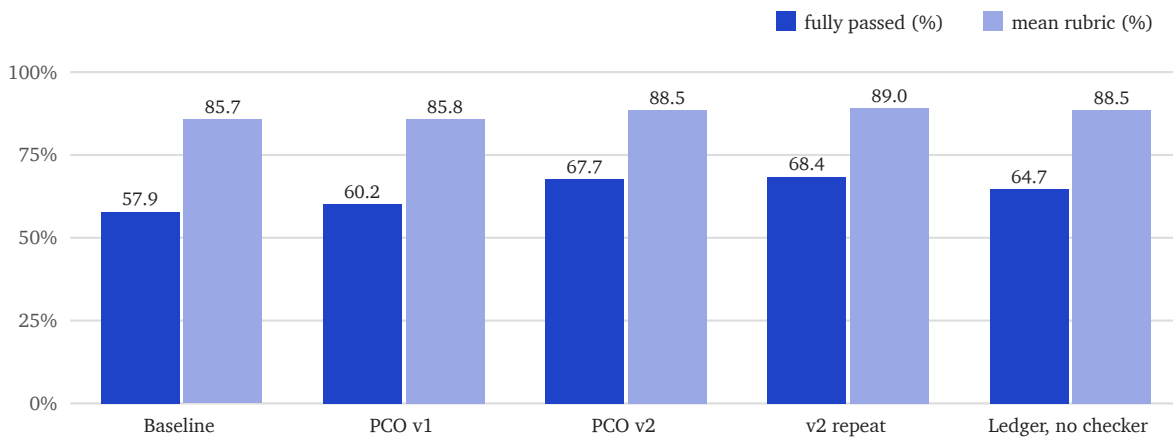



Figure 1. iOSWorld (133 tasks): share of tasks with every rubric criterion satisfied, and mean rubric score, per configuration.

**Table 1. iOSWorld, 133 tasks, rubric judge. Cost is the agent's (CLI estimate).**

| Configuration | Fully passed | Rate | 95% CI | Mean rubric | 95% CI | Cost |
|---|---|---|---|---|---|---|
| Baseline (Claude Code + Verdikt tools) | 77/133 | 57.9% | 49.4–66.0% | 0.857 | 0.820–0.892 | $123 |
| PCO v1 | 80/133 | 60.2% | 51.7–68.1% | 0.858 | 0.820–0.893 | $176 |
| PCO v2 | 90/133 | 67.7% | 59.3–75.0% | 0.885 | 0.849–0.918 | $186 |
| PCO v2, repeat | 91/133 | 68.4% | 60.1–75.7% | 0.890 | 0.855–0.923 | $181 |
| v2 ledger, no checker (ablation) | 86/133 | 64.7% | 56.2–72.3% | 0.885 | 0.851–0.918 | $141 |

**Table 2. Paired comparisons on the same tasks. "+a / –b": tasks fully passed only by the second / only by the first configuration.**

| First vs second | Tasks | + / – | McNemar p | Mean rubric Δ (95% CI) | Sign test p |
|---|---|---|---|---|---|
| Baseline vs PCO v2 | 133 | +17 / –4 | 0.007 | +0.028 (+0.005 to +0.052) | 0.003 |
| Baseline vs PCO v2 repeat | 133 | +19 / –5 | 0.007 | +0.033 (+0.011 to +0.057) | 0.003 |
| PCO v2 vs its repeat | 133 | +10 / –9 | 1.000 | +0.005 (-0.018 to +0.028) | 0.377 |
| Baseline vs PCO v1 | 133 | +14 / –11 | 0.690 | +0.001 (-0.028 to +0.029) | 0.576 |
| Ledger only vs PCO v2 | 133 | +14 / –10 | 0.541 | +0.000 (-0.027 to +0.026) | 0.749 |

PCO v2 is the only configuration with a significant gain over the baseline, and an independent repeat lands within one task of it. Version 1 is indistinguishable from the baseline, which locates the effect in the v2 changes rather than in decomposition as such. The gain is concentrated in multi-app tasks (25 to 36 of 60 fully passed); memory tasks moved from 28 to 29 of 46; single-app tasks moved from 24 to 25 of 27. Personal-memory tasks depend on knowing which app holds which part of the user's life, which PCO does not supply.

**Table 3. iOSWorld by category, baseline vs PCO v2.**

| Category | Tasks | Baseline passed | Baseline mean | PCO v2 passed | PCO v2 mean |
|---|---|---|---|---|---|
| memory | 46 | 28 | 0.851 | 29 | 0.845 |
| multi-app | 60 | 25 | 0.807 | 36 | 0.870 |
| single-app | 27 | 24 | 0.979 | 25 | 0.989 |

**The checker at work.** In the v2 run the agent recorded 759 obligations (435 KNOW, 249 DO, 65 SAY, 10 IF); 734 ended proven, 25 closed as absent after a challenge, and 26 tasks were sent back by the checker at least once. **Case study.** On task multi-081 (read design feedback in a chat app, update a document and a slide, set a timer) the baseline agent found no feedback in the channel and changed nothing (rubric 0.125); under PCO it recorded four obligations, found the feedback in the thread replies and proved each (rubric 0.875).

### 6.2 Ablation

The ablation keeps v2's ledger prompt but never sends the agent back (zero checker rounds). It fully passed 86/133 (mean rubric 0.885) against 90/133 (0.885) with the checker: on the same tasks the checker version passed 14 tasks the ledger-only version did not and lost 10 (McNemar $p = 0.541$). Against the baseline, the ledger prompt alone gained 16 and lost 7 ($p = 0.093$). Together these separate what the obligation list contributes from what the checker's refusals contribute.

### 6.3 AndroidWorld

**Table 4. AndroidWorld, official grader, pass@1.**

| Configuration | Seed | Passed | Rate | 95% CI |
|---|---|---|---|---|
| A | 30 | 110/116 | 94.8% | 89.2–97.6% |
| A | 7 | 110/116 | 94.8% | 89.2–97.6% |
| A | 8 | 111/116 | 95.7% | 90.3–98.2% |
| B | 30 | 114/116 | 98.3% | 93.9–99.5% |
| B (seeds 7+8) | 30 | 113/116 | 97.4% | 92.7–99.1% |
| C (contracts) | 30 | 113/116 | 97.4% | 92.7–99.1% |
| C (contracts) | 7 | 110/116 | 94.8% | 89.2–97.6% |
| C (contracts) | 8 | 109/116 | 94.0% | 88.1–97.0% |

The operator alone averages 95.1% over three suite seeds. On seed 30, learned playbooks and executable contracts reached 97–98% (5 tasks gained and 2 lost for contracts, $p = 0.453$), but the contracts' gain did not repeat on seeds 7 and 8. Across suite seeds, contracts scored 113/116 (seed 30), 110/116 (seed 7), 109/116 (seed 8) versus the operator alone at 110/116 (seed 30), 110/116 (seed 7), 111/116 (seed 8). Averaged over seeds, contracts score 95.4% against 95.1%: near the ceiling the differences are run-to-run variation, which is why the main evaluation uses iOSWorld.

## 7 Discussion

**What drives the gain.** Two results locate the effect. Version 1, with a checker but an incomplete ledger, is indistinguishable from the baseline: a checker can only certify the parts the agent thought of. The ablation, with v2's complete ledger but no resumption rounds, recovers nearly all of v2's gain: once every requested action, fact and derived result is an obligation the agent must close with evidence from a screen, the agent finishes them in the first pass, and the few tasks the gate sends back rarely have enough budget left to change the outcome. The checker's remaining role is verification rather than steering: it rejects claims the screen does not show, completes the answer only from proven facts, and produces the certificate. **What remains.** Of the 43 tasks v2 did not fully pass, 13 used their whole action budget; the rest mostly missed a rubric item that credits consulting a specific app, a form of app knowledge the method deliberately does not supply. **Cost.**

PCO v2 cost $186 for 133 tasks against $123 for the baseline (agent cost, CLI estimate); the ledger without resumption rounds reached the same mean rubric for $141, so in practice the gate is worth running only where its audit value matters. **Mechanical and human verification.** The checker can only certify what a screen shows; irreversible or consequential actions call for the complementary mechanism of [13], authority keyed to reversibility, where such actions are gated to a human.

## 8 Limitations and threats to validity

One model family. The iOSWorld judge is Claude Sonnet 4.6 rather than the default GPT-5.4-mini, used for every run alike; a model judge can err. Five apps were built with a minimum iOS version of 15.0 because Xcode 27 rejects 13 and 14. Running five simulators in parallel exposed a driver bug in which two sessions could share a companion port and one task's actions reached another's simulator; we fixed it, identified every task whose session started within 15 s of another, and re-ran those tasks for the baseline and v1 (originals archived). The method was developed while looking at failures on these benchmarks, and the v2 changes were designed after inspecting v1's failures on the same 133 iOSWorld tasks; the changes are generic (they name no app, task or value), v2 was evaluated twice, and the ablation isolates its parts, but a held-out benchmark is the proper test and is future work. AndroidWorld ran on arm64, and the runner completes setup steps the official script misses without touching graders. The iOS runs used the product's iOS session; the open-source driver is a re-implementation of the same calls.

## 9 Conclusion

Making completion checkable changes what an agent does: it finishes the parts it used to skip and reports only what it can show. The effect depends on the ledger being complete, which the v2 design achieves without app knowledge, and every run leaves an auditable record, which matters as much in software testing as on benchmarks.

## Code and data availability

The method (check language, obligation ledger and checker, stable targeting, device drivers and a reference MCP server) is released as Verdikt Core under the Apache-2.0 license at https://github.com/verdikt-ai/verdikt. Every trajectory, screen, ledger and certificate behind the tables, the per-task results as CSV and JSON, and the statistics script are published at https://verdikt-ai.github.io/verdikt/pco/results, where each run can be replayed step by step.

## References


1. Rawles et al. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. arXiv:2405.14573, 2024.
2. Jang et al. iOSWorld: A Benchmark for Personally Intelligent Phone Agents. arXiv:2606.09764, 2026.
3. AndroidWorld public leaderboard (self-reported results).
4. Favreau et al. Do Multi-Agents Dream of Electric Screens? Achieving Perfect Accuracy on AndroidWorld Through Task Decomposition. arXiv:2602.07787, 2026; trajectories at minitap.ai/benchmark.
5. Mobilerun benchmark methodology, mobilerun.ai/benchmark/method, 2026.
6. Sun et al. AdaPlanner: Adaptive Planning from Feedback with Language Models. NeurIPS 2023.
7. WebTestPilot. arXiv:2602.11724, 2026.
8. VeriSafe Agent. MobiCom 2025, arXiv:2503.18492.
9. Surfer 2. arXiv:2510.19949, 2025.
10. AdecPilot. arXiv:2604.07767, 2026.

11. Wang et al. Mobile-Agent-E. arXiv:2501.11733, 2025.
12. Necula. Proof-Carrying Code. POPL 1997.
13. Haldankar, K. R. and Mohamed, H. A. Event-Driven Coordination of Episodic AI Agents: An Architecture for Cost-Bounded, Auditable, Human-Authoritative Multi-Agent Work. Preprint, version 2. Zenodo, 2026. doi:10.5281/zenodo.23084582.